```
# ===== Import các thư viện cần dùng =====
import warnings
import numpy as np                      # tính toán số học
import pandas as pd                     # xử lý bảng dữ liệu
import matplotlib.pyplot as plt         # vẽ biểu đồ
import seaborn as sns                   # biểu đồ thống kê đẹp hơn
from scipy import stats                 # các phép kiểm định thống
kê (Shapiro, Chi-square, KS)
from sklearn.preprocessing import MinMaxScaler, StandardScaler   #
chuẩn hóa dữ liệu
from sklearn.decomposition import PCA                            #
giảm chiều dữ liệu

warnings.filterwarnings("ignore")       # ẩn cảnh báo cho gọn
sns.set_theme(style="whitegrid")        # giao diện nền lưới cho
biểu đồ
%matplotlib inline
pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")   # hiển
thị số 3 chữ số thập phân

# ===== CẤU HÌNH: sửa cho đúng với dữ liệu của bạn =====
TRAIN_PATH = "train.csv"      # đường dẫn file train
TEST_PATH = "test.csv"        # đường dẫn file test
TARGET = "SalePrice"          # tên cột giá nhà (chỉ có trong train)
ID_COL = "Id"                 # cột mã định danh; nếu không có thì
đặt None
DISCRETE_MAX_UNIQUE = 20      # cột số có <= 20 giá trị khác nhau sẽ
coi là rời rạc (discrete)

# ===== Đọc dữ liệu =====
train = pd.read_csv(TRAIN_PATH)
test = pd.read_csv(TEST_PATH)

print("Kích thước train:", train.shape)
print("Kích thước test :", test.shape)

# Test không có cột giá nhà (vì đó là thứ cần dự đoán). Kiểm tra lại
cho chắc:
print("Cột có ở train nhưng không có ở test:", set(train.columns) -
set(test.columns))

train.head()
```

```
Kích thước train: (1460, 81)
Kích thước test : (1459, 80)
Cột có ở train nhưng không có ở test: {'SalePrice'}
```

```
   Id  MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape
\
0   1          60       RL       65.000     8450   Pave   NaN      Reg
```

```
1   2          20       RL       80.000     9600   Pave   NaN      Reg

2   3          60       RL       68.000    11250   Pave   NaN      IR1

3   4          70       RL       60.000     9550   Pave   NaN      IR1

4   5          60       RL       84.000    14260   Pave   NaN      IR1


  LandContour Utilities LotConfig LandSlope Neighborhood Condition1  \
0         Lvl    AllPub    Inside       Gtl      CollgCr       Norm
1         Lvl    AllPub       FR2       Gtl      Veenker      Feedr
2         Lvl    AllPub    Inside       Gtl      CollgCr       Norm
3         Lvl    AllPub    Corner       Gtl      Crawfor       Norm
4         Lvl    AllPub       FR2       Gtl      NoRidge       Norm

  Condition2 BldgType HouseStyle  OverallQual  OverallCond  YearBuilt
\
0       Norm     1Fam     2Story            7            5       2003

1       Norm     1Fam     1Story            6            8       1976

2       Norm     1Fam     2Story            7            5       2001

3       Norm     1Fam     2Story            7            5       1915

4       Norm     1Fam     2Story            8            5       2000


   YearRemodAdd RoofStyle RoofMatl Exterior1st Exterior2nd  ...
Fireplaces  \
0          2003     Gable  CompShg     VinylSd     VinylSd  ...
0
1          1976     Gable  CompShg     MetalSd     MetalSd  ...
1
2          2002     Gable  CompShg     VinylSd     VinylSd  ...
1
3          1970     Gable  CompShg     Wd Sdng     Wd Shng  ...
1
4          2000     Gable  CompShg     VinylSd     VinylSd  ...
1

   FireplaceQu GarageType GarageYrBlt GarageFinish GarageCars
GarageArea  \
0          NaN     Attchd   2,003.000          RFn          2
548
1           TA     Attchd   1,976.000          RFn          2
460
2           TA     Attchd   2,001.000          RFn          2
608
```

```
3           Gd     Detchd   1,998.000          Unf          3
642
4           TA     Attchd   2,000.000          RFn          3
836

  GarageQual GarageCond  PavedDrive WoodDeckSF  OpenPorchSF
EnclosedPorch  \
0         TA         TA           Y          0           61
0
1         TA         TA           Y        298            0
0
2         TA         TA           Y          0           42
0
3         TA         TA           Y          0           35
272
4         TA         TA           Y        192           84
0

   3SsnPorch ScreenPorch PoolArea PoolQC Fence  MiscFeature  MiscVal
MoSold  \
0           0           0        0    NaN   NaN          NaN        0
2
1           0           0        0    NaN   NaN          NaN        0
5
2           0           0        0    NaN   NaN          NaN        0
9
3           0           0        0    NaN   NaN          NaN        0
2
4           0           0        0    NaN   NaN          NaN        0
12

   YrSold  SaleType  SaleCondition  SalePrice
0    2008        WD         Normal     208500
1    2007        WD         Normal     181500
2    2008        WD         Normal     223500
3    2006        WD        Abnorml     140000
4    2008        WD         Normal     250000

[5 rows x 81 columns]

# ===== Tách cột thành 2 nhóm: sô´ (numeric) và phân loại
(categorical) =====
# Loại bo' cột mục tiêu và cột Id vì chúng không pha'i đặc trưng
(feature)
feature_cols = [c for c in train.columns if c not in [TARGET, ID_COL]]

# Cột có kiê'u sô´ -> numeric; còn lại -> categorical
num_cols = [c for c in feature_cols if
pd.api.types.is_numeric_dtype(train[c])]
cat_cols = [c for c in feature_cols if c not in num_cols]
```

```
print(f"Sô´đặc trưng: {len(feature_cols)} (numeric: {len(num_cols)},
categorical: {len(cat_cols)})")
```

```
Sô´đặc trưng: 79 (numeric: 36, categorical: 43)
```

1. Xác định tính chất dữ liệu *(Mục 2.1.1 và 2.4)*

Theo Chương 2, cần xác định:

- **Dạng dữ liệu:** structured (bảng), unstructured hay semi-structured.
- **Loại thuộc tính:** categorical (nominal, ordinal) hay numeric (interval, ratio); rời rạc hay liên tục.
- **Số biến:** univariate (1 biến), bivariate (2 biến), multivariate (từ 3 biến trở lên).

```
# ===== Bảng tính chất của từng cột =====
rows = []
for c in feature_cols:
    s = train[c]
    if c in num_cols:
        # Cột số ít giá trị khác nhau -> rời rạc (vd: số phòng);
nhiều -> liên tục (vd: diện tích)
        loai = "Numeric - rời rạc" if s.nunique() <=
DISCRETE_MAX_UNIQUE else "Numeric - liên tục"
        thang_do = "Interval / Ratio"
    else:
        loai = "Categorical"
        thang_do = "Nominal / Ordinal"   # cần đọc mô tả cột để
biết có thứ tự hay không
    rows.append({
        "feature": c,
        "dtype": str(s.dtype),
        "loại dữ liệu": loai,
        "thang đo": thang_do,
        "số giá trị khác nhau": s.nunique(),
        "ví dụ": s.dropna().iloc[0] if s.notna().any() else None,
    })

properties = pd.DataFrame(rows).set_index("feature")
properties
```

```
                dtype        loại dữ liệu           thang đo  \
feature
MSSubClass      int64   Numeric - rời rạc   Interval / Ratio
MSZoning          str         Categorical  Nominal / Ordinal
LotFrontage   float64  Numeric - liên tục   Interval / Ratio
LotArea         int64  Numeric - liên tục   Interval / Ratio
Street            str         Categorical  Nominal / Ordinal
...               ...                 ...                ...
MiscVal         int64  Numeric - liên tục   Interval / Ratio
```

```
MoSold           int64   Numeric - rời rạc   Interval / Ratio
YrSold           int64   Numeric - rời rạc   Interval / Ratio
SaleType           str         Categorical  Nominal / Ordinal
SaleCondition      str         Categorical  Nominal / Ordinal

               số giá trị khác nhau   ví dụ
feature
MSSubClass                       15      60
MSZoning                          5      RL
LotFrontage                     110  65.000
LotArea                        1073    8450
Street                            2    Pave
...                             ...     ...
MiscVal                          21       0
MoSold                           12       2
YrSold                            5    2008
SaleType                          9      WD
SaleCondition                     6  Normal

[79 rows x 5 columns]
```

```python
# ===== Thống kê nhanh về tính chất tổng thể =====
print("Dạng dữ liệu       : structured data (dữ liệu bảng / record
data)")
print("Số biến            : multivariate (nhiều hơn 2 biến)")
print(f"Số mẫu (dòng)      : {len(train)}")
print(f"Số dòng trùng lặp  : {train.duplicated().sum()}")
print()
print("Phân bố loại dữ liệu:")
print(properties["loại dữ liệu"].value_counts())
```

```
Dạng dữ liệu       : structured data (dữ liệu bảng / record data)
Số biến            : multivariate (nhiều hơn 2 biến)
Số mẫu (dòng)      : 1460
Số dòng trùng lặp  : 0

Phân bố loại dữ liệu:
loại dữ liệu
Categorical           43
Numeric - liên tục    20
Numeric - rời rạc     16
Name: count, dtype: int64
```

Nhận xét (điền sau khi chạy):

- Dữ liệu là dạng bảng (structured), gồm … biến numeric và … biến categorical.
- Các biến như `OverallQual` (thang 1–10) là *ordinal* dù được lưu bằng số; biến `Neighborhood` là *nominal*.
- Biến mục tiêu `SalePrice` là numeric liên tục (ratio) → đây là bài toán **hồi quy**.

2. Phân tích đơn biến *(Mục 2.5)*

Với mỗi biến, mô tả bằng 4 nhóm đặc trưng:

1. **Phân phối tần số** (histogram, bar chart)
2. **Xu hướng trung tâm** (mean, median, mode)
3. **Độ phân tán** (range, variance, std, IQR, MAD, CV)
4. **Hình dạng** (skewness, kurtosis)

```
# ===== Hàm tính bảng thống kê mô tả cho nhiều cột số =====
def describe_numeric(df, cols):
    out = []
    for c in cols:
        s = df[c].dropna()                          # bỏ giá trị thiếu
trước khi tính
        if s.empty:
            continue
        q1, q3 = s.quantile([0.25, 0.75])           # tứ phân vị thứ
nhất và thứ ba
        mode = s.mode()                             # mode: giá trị xuất
hiện nhiều nhất
        out.append({
            "feature": c,
            "count": s.count(),
            "mean": s.mean(),                       # trung bình
            "median": s.median(),                   # trung vị
            "mode": mode.iloc[0] if not mode.empty else np.nan,
            "min": s.min(),
            "Q1": q1,
            "Q3": q3,
            "max": s.max(),
            "range": s.max() - s.min(),             # khoảng biến thiên
= max - min
            "IQR": q3 - q1,                         # khoảng tứ phân vị
(Eq. 2.9)
            "variance": s.var(),                    # phương sai
            "std": s.std(),                         # độ lệch chuẩn (Eq.
2.8, chia N-1)
            "MAD": (s - s.mean()).abs().mean(),     # độ lệch tuyệt đối
trung bình (Eq. 2.16)
            "CV(%)": s.std() / s.mean() * 100 if s.mean() != 0 else
np.nan,  # hệ số biến thiên
            "skewness": s.skew(),                   # độ lệch (Eq. 2.13)
            "kurtosis": s.kurt(),                   # độ nhọn (Eq. 2.14)
        })
    return pd.DataFrame(out).set_index("feature")
```

2.1 Biến mục tiêu – giá nhà

```
# ===== Thô´ng kê mô tả cu̇a giá nhà =====
describe_numeric(train, [TARGET]).T
```

feature	SalePrice
count	1,460.000
mean	180,921.196
median	163,000.000
mode	140,000.000
min	34,900.000
Q1	129,975.000
Q3	214,000.000
max	755,000.000
range	720,100.000
IQR	84,025.000
variance	6,311,111,264.297
std	79,442.503
MAD	57,434.770
CV(%)	43.910
skewness	1.883
kurtosis	6.536

```
# ===== Vẽ 4 biểu đồ` cho biế´n mục tiêu: histogram, boxplot, Q-Q
plot (gô´c và sau log) =====
fig, axes = plt.subplots(1, 4, figsize=(20, 4))

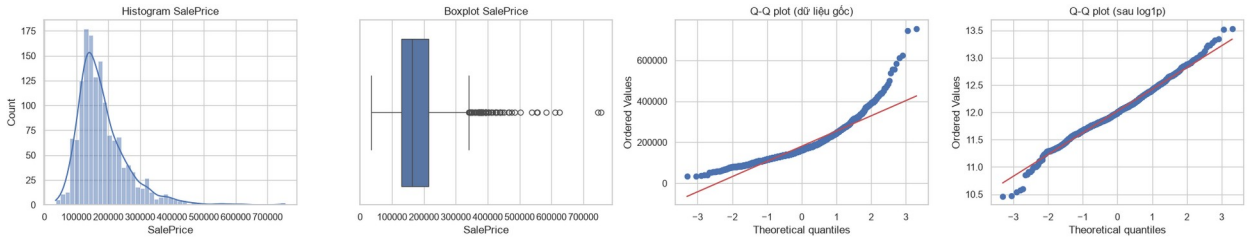
sns.histplot(train[TARGET], kde=True, ax=axes[0])        # histogram +
đường mật độ
axes[0].set_title(f"Histogram {TARGET}")

sns.boxplot(x=train[TARGET], ax=axes[1])                 # boxplot:
thấ´y median, IQR và outlier
axes[1].set_title(f"Boxplot {TARGET}")

# Q-Q plot: nê´u dữ liệu chuẩn thì các điểm nằm sát đường thẳng
(Hình 2.10)
stats.probplot(train[TARGET].dropna(), dist="norm", plot=axes[2])
axes[2].set_title("Q-Q plot (dữ liệu gôć)")

# Biế´n đổi log1p = log(1 + x) để giảm độ lệch phải
stats.probplot(np.log1p(train[TARGET].dropna()), dist="norm",
plot=axes[3])
axes[3].set_title("Q-Q plot (sau log1p)")

plt.tight_layout()
plt.show()
```




```
# ===== Kiểm tra độ lệch và tính chuẩn =====
sk = train[TARGET].skew()
print(f"Skewness của {TARGET}: {sk:.3f} -> {'lệch phải (positive
skew)' if sk > 0 else 'lệch trái (negative skew)'}")
print(f"Mean = {train[TARGET].mean():,.0f} | Median =
{train[TARGET].median():,.0f}")
print("Lệch phải thì mean > median (vài căn nhà rất đắt kéo trung bình
lên).")
print(f"Skewness sau log1p: {np.log1p(train[TARGET]).skew():.3f}  (gần
0 nghĩa là gần đối xứng)")

# Kiểm định Shapiro-Wilk: H0 = dữ liệu có phân phối chuẩn. p < 0.05
-> bác bỏ H0
sample = train[TARGET].dropna().sample(min(5000, len(train)),
random_state=0)
stat, p = stats.shapiro(sample)
print(f"\nShapiro-Wilk (dữ liệu gốc): p = {p:.4g} -> {'KHÔNG' if p <
0.05 else ''} phân phối chuẩn")
stat, p = stats.shapiro(np.log1p(sample))
print(f"Shapiro-Wilk (sau log1p)  : p = {p:.4g}")
```

```
Skewness của SalePrice: 1.883 -> lệch phải (positive skew)
Mean = 180,921 | Median = 163,000
Lệch phải thì mean > median (vài căn nhà rất đắt kéo trung bình lên).
Skewness sau log1p: 0.121  (gần 0 nghĩa là gần đối xứng)

Shapiro-Wilk (dữ liệu gốc): p = 3.206e-33 -> KHÔNG phân phối chuẩn
Shapiro-Wilk (sau log1p)  : p = 1.149e-07
```

Nhận xét (điền sau khi chạy): giá nhà lệch phải, có đuôi dài bên phải (nhà cao cấp). Sau khi log-transform, phân phối gần chuẩn hơn → nên dự đoán `log(SalePrice)` để mô hình hoạt động tốt hơn.

> Lưu ý: khi **positive skew** thì *mean > median*. (Một số đoạn trong sách diễn đạt chưa nhất quán chỗ này, bạn nên theo quy ước vừa nêu.)

2.2 Các biến numeric

```
# ===== Bảng thống kê mô tả cho tất cả biến numeric =====
stats_num = describe_numeric(train, num_cols)
stats_num
```

feature	count	mean	median	mode	min	Q1
MSSubClass	1460	56.897	50.000	20.000	20.000	20.000
LotFrontage	1201	70.050	69.000	60.000	21.000	59.000
LotArea	1460	10,516.828	9,478.500	7,200.000	1,300.000	7,553.500
OverallQual	1460	6.099	6.000	5.000	1.000	5.000
OverallCond	1460	5.575	5.000	5.000	1.000	5.000
YearBuilt	1460	1,971.268	1,973.000	2,006.000	1,872.000	1,954.000
YearRemodAdd	1460	1,984.866	1,994.000	1,950.000	1,950.000	1,967.000
MasVnrArea	1452	103.685	0.000	0.000	0.000	0.000
BsmtFinSF1	1460	443.640	383.500	0.000	0.000	0.000
BsmtFinSF2	1460	46.549	0.000	0.000	0.000	0.000
BsmtUnfSF	1460	567.240	477.500	0.000	0.000	223.000
TotalBsmtSF	1460	1,057.429	991.500	0.000	0.000	795.750
1stFlrSF	1460	1,162.627	1,087.000	864.000	334.000	882.000
2ndFlrSF	1460	346.992	0.000	0.000	0.000	0.000
LowQualFinSF	1460	5.845	0.000	0.000	0.000	0.000
GrLivArea	1460	1,515.464	1,464.000	864.000	334.000	1,129.500
BsmtFullBath	1460	0.425	0.000	0.000	0.000	0.000
BsmtHalfBath	1460	0.058	0.000	0.000	0.000	0.000
FullBath	1460	1.565	2.000	2.000	0.000	1.000
HalfBath	1460	0.383	0.000	0.000	0.000	0.000
BedroomAbvGr	1460	2.866	3.000	3.000	0.000	2.000
KitchenAbvGr	1460	1.047	1.000	1.000	0.000	1.000
TotRmsAbvGrd	1460	6.518	6.000	6.000	2.000	5.000

```
Fireplaces      1460      0.613     1.000     0.000     0.000
0.000
GarageYrBlt     1379  1,978.506 1,980.000 2,005.000 1,900.000
1,961.000
GarageCars      1460      1.767     2.000     2.000     0.000
1.000
GarageArea      1460    472.980   480.000     0.000     0.000
334.500
WoodDeckSF      1460     94.245     0.000     0.000     0.000
0.000
OpenPorchSF     1460     46.660    25.000     0.000     0.000
0.000
EnclosedPorch   1460     21.954     0.000     0.000     0.000
0.000
3SsnPorch       1460      3.410     0.000     0.000     0.000
0.000
ScreenPorch     1460     15.061     0.000     0.000     0.000
0.000
PoolArea        1460      2.759     0.000     0.000     0.000
0.000
MiscVal         1460     43.489     0.000     0.000     0.000
0.000
MoSold          1460      6.322     6.000     6.000     1.000
5.000
YrSold          1460  2,007.816 2,008.000 2,009.000 2,006.000
2,007.000

                        Q3         max       range       IQR
variance  \
feature

MSSubClass        70.000     190.000     170.000    50.000
1,789.338
LotFrontage       80.000     313.000     292.000    21.000
589.749
LotArea       11,601.500 215,245.000 213,945.000 4,048.000
99,625,649.650
OverallQual        7.000      10.000       9.000     2.000
1.913
OverallCond        6.000       9.000       8.000     1.000
1.238
YearBuilt      2,000.000   2,010.000     138.000    46.000
912.215
YearRemodAdd   2,004.000   2,010.000      60.000    37.000
426.233
MasVnrArea       166.000   1,600.000   1,600.000   166.000
32,784.971
BsmtFinSF1       712.250   5,644.000   5,644.000   712.250
208,025.468
BsmtFinSF2         0.000   1,474.000   1,474.000     0.000
```

```
26,023.908
BsmtUnfSF         808.000   2,336.000   2,336.000   585.000
195,246.406
TotalBsmtSF     1,298.250   6,110.000   6,110.000   502.500
192,462.362
1stFlrSF        1,391.250   4,692.000   4,358.000   509.250
149,450.079
2ndFlrSF          728.000   2,065.000   2,065.000   728.000
190,557.075
LowQualFinSF        0.000     572.000     572.000     0.000
2,364.204
GrLivArea       1,776.750   5,642.000   5,308.000   647.250
276,129.633
BsmtFullBath        1.000       3.000       3.000     1.000
0.269
BsmtHalfBath        0.000       2.000       2.000     0.000
0.057
FullBath            2.000       3.000       3.000     1.000
0.304
HalfBath            1.000       2.000       2.000     1.000
0.253
BedroomAbvGr        3.000       8.000       8.000     1.000
0.665
KitchenAbvGr        1.000       3.000       3.000     0.000
0.049
TotRmsAbvGrd        7.000      14.000      12.000     2.000
2.642
Fireplaces          1.000       3.000       3.000     1.000
0.416
GarageYrBlt     2,002.000   2,010.000     110.000    41.000
609.583
GarageCars          2.000       4.000       4.000     1.000
0.558
GarageArea        576.000   1,418.000   1,418.000   241.500
45,712.510
WoodDeckSF        168.000     857.000     857.000   168.000
15,709.813
OpenPorchSF        68.000     547.000     547.000    68.000
4,389.861
EnclosedPorch       0.000     552.000     552.000     0.000
3,735.550
3SsnPorch           0.000     508.000     508.000     0.000
859.506
ScreenPorch         0.000     480.000     480.000     0.000
3,108.889
PoolArea            0.000     738.000     738.000     0.000
1,614.216
MiscVal             0.000  15,500.000  15,500.000     0.000
246,138.055
MoSold              8.000      12.000      11.000     3.000
```

```
7.310
YrSold          2,009.000   2,010.000        4.000     2.000
1.764

                      std        MAD      CV(%)  skewness  kurtosis
feature
MSSubClass         42.301     31.283     74.346     1.408     1.580
LotFrontage        24.285     16.762     34.668     2.164    17.453
LotArea         9,981.265  3,758.814     94.908    12.208   203.243
OverallQual         1.383      1.098     22.675     0.217     0.096
OverallCond         1.113      0.889     19.959     0.693     1.106
YearBuilt          30.203     25.067      1.532    -0.613    -0.440
YearRemodAdd       20.645     18.623      1.040    -0.504    -1.272
MasVnrArea        181.066    129.782    174.631     2.669    10.082
BsmtFinSF1        456.098    367.370    102.808     1.686    11.118
BsmtFinSF2        161.319     82.535    346.556     4.255    20.113
BsmtUnfSF         441.867    353.282     77.898     0.920     0.475
TotalBsmtSF       438.705    321.284     41.488     1.524    13.250
1stFlrSF          386.588    300.576     33.251     1.377     5.746
2ndFlrSF          436.528    396.478    125.803     0.813    -0.553
LowQualFinSF       48.623     11.481    831.943     9.011    83.235
GrLivArea         525.480    397.325     34.675     1.367     4.895
BsmtFullBath        0.519      0.499    121.998     0.596    -0.839
BsmtHalfBath        0.239      0.109    414.975     4.103    16.397
FullBath            0.551      0.522     35.201     0.037    -0.857
HalfBath            0.503      0.479    131.344     0.676    -1.077
BedroomAbvGr        0.816      0.576     28.460     0.212     2.231
KitchenAbvGr        0.220      0.090     21.053     4.488    21.532
TotRmsAbvGrd        1.625      1.280     24.938     0.676     0.881
Fireplaces          0.645      0.579    105.163     0.650    -0.217
GarageYrBlt        24.690     20.913      1.248    -0.649    -0.418
GarageCars          0.747      0.584     42.290    -0.343     0.221
GarageArea        213.805    160.019     45.204     0.180     0.917
WoodDeckSF        125.339    101.996    132.993     1.541     2.993
OpenPorchSF        66.256     47.678    141.997     2.364     8.490
EnclosedPorch      61.119     37.660    278.395     3.090    10.431
3SsnPorch          29.317      6.707    859.849    10.304   123.662
ScreenPorch        55.757     27.729    370.212     4.122    18.439
PoolArea           40.177      5.491  1,456.278    14.828   223.268
MiscVal           496.123     83.880  1,140.800    24.477   701.003
MoSold              2.704      2.143     42.766     0.212    -0.404
YrSold              1.328      1.149      0.066     0.096    -1.191
```

```
# ===== Top biến lệch nhiều nhất và biến có độ nhọn lớn nhất
=====
print("Top 10 biến lệch nhất (|skewness| lớn):")
print(stats_num["skewness"].abs().sort_values(ascending=False).head(10
))
print("\nTop 10 biến có kurtosis lớn nhất (đuôi nặng, nhiều
```

```
outlier):")
print(stats_num["kurtosis"].sort_values(ascending=False).head(10))
```

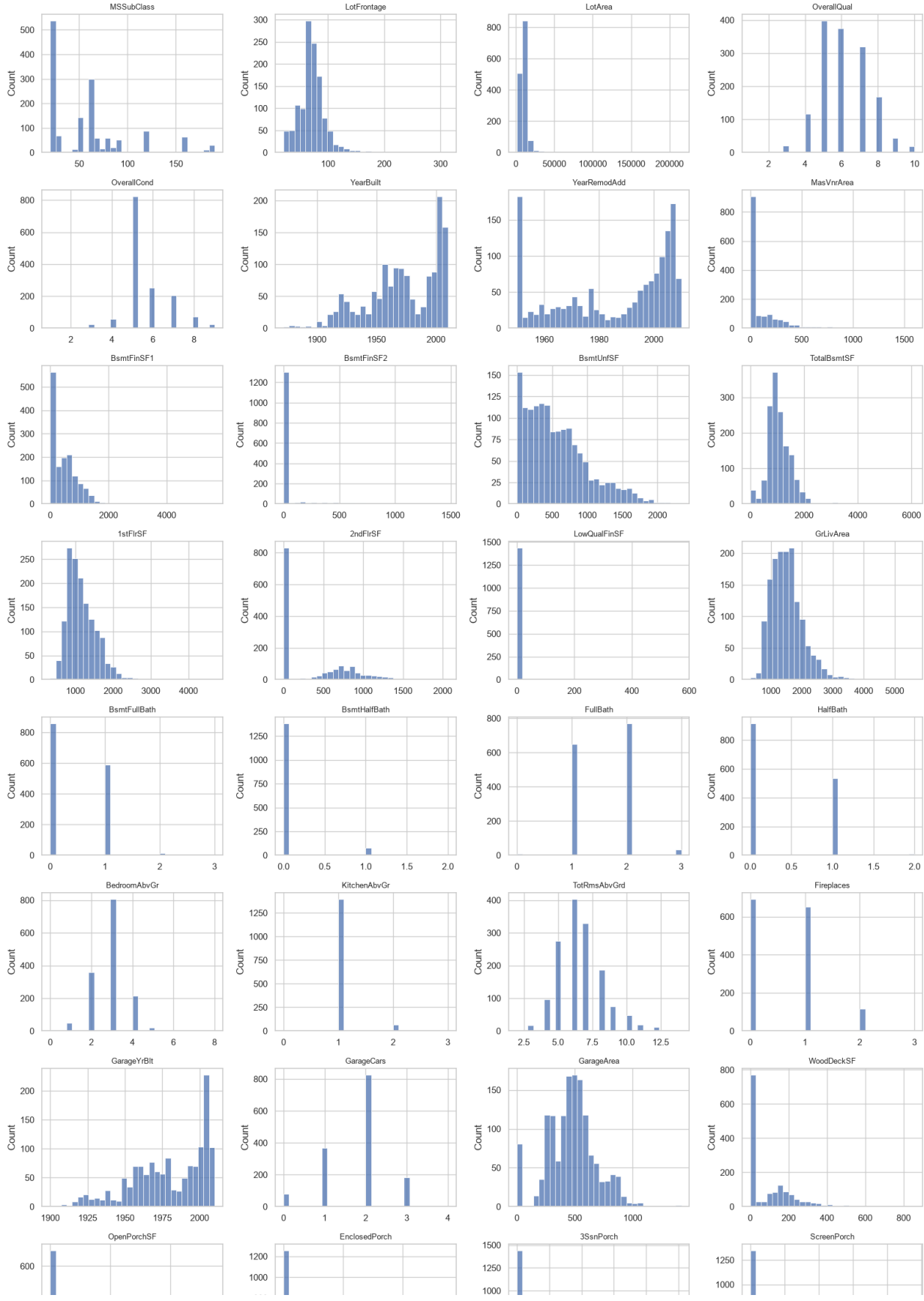
```
Top 10 biến lệch nhất (|skewness| lớn):
feature
MiscVal         24.477
PoolArea        14.828
LotArea         12.208
3SsnPorch       10.304
LowQualFinSF     9.011
KitchenAbvGr     4.488
BsmtFinSF2       4.255
ScreenPorch      4.122
BsmtHalfBath     4.103
EnclosedPorch    3.090
Name: skewness, dtype: float64

Top 10 biến có kurtosis lớn nhất (đuôi nặng, nhiều outlier):
feature
MiscVal        701.003
PoolArea       223.268
LotArea        203.243
3SsnPorch      123.662
LowQualFinSF    83.235
KitchenAbvGr    21.532
BsmtFinSF2      20.113
ScreenPorch     18.439
LotFrontage     17.453
BsmtHalfBath    16.397
Name: kurtosis, dtype: float64
```

```
# ===== Histogram cuả tấ t caả biế n numeric (lưới nhiề u ô con)
=====
n = len(num_cols)
ncols = 4
nrows = int(np.ceil(n / ncols))
fig, axes = plt.subplots(nrows, ncols, figsize=(4 * ncols, 3 * nrows))
axes = np.atleast_1d(axes).ravel()

for ax, c in zip(axes, num_cols):
    sns.histplot(train[c].dropna(), bins=30, ax=ax)
    ax.set_title(c, fontsize=9)
    ax.set_xlabel("")
for ax in axes[n:]:          # tă t các ô thừa
    ax.axis("off")

plt.tight_layout()
plt.show()
```


MSSubClass
LotFrontage
LotArea
OverallQual
OverallCond
YearBuilt
YearRemodAdd
MasVnrArea
BsmtFinSF1
BsmtFinSF2
BsmtUnfSF
TotalBsmtSF
1stFlrSF
2ndFlrSF
LowQualFinSF
GrLivArea
BsmtFullBath
BsmtHalfBath
FullBath
HalfBath
BedroomAbvGr
KitchenAbvGr
TotRmsAbvGrd
Fireplaces
GarageYrBlt
GarageCars
GarageArea
WoodDeckSF
OpenPorchSF
EnclosedPorch
3SsnPorch
ScreenPorch
Count

2.3 Các biến categorical

```
# ===== Thô´ng kê biê´n categorical: sô´ nhóm, mode và tỷ lệ cu̇a
mode =====
cat_summary = []
for c in cat_cols:
    vc = train[c].value_counts(dropna=False)
    cat_summary.append({
        "feature": c,
        "sô´nhóm": train[c].nunique(),
        "mode": vc.index[0],                          # nhóm xuâ´t
hiện nhiê`u nhâ´t
        "tỷ lệ mode (%)": vc.iloc[0] / len(train) * 100,
    })
cat_summary = pd.DataFrame(cat_summary).set_index("feature")
cat_summary.sort_values("tỷ lệ mode (%)", ascending=False)
```

feature	sô´nhóm	mode	tỷ lệ mode (%)
Utilities	2	AllPub	99.932
Street	2	Pave	99.589
PoolQC	3	NaN	99.521
Condition2	8	Norm	98.973
RoofMatl	8	CompShg	98.219
Heating	6	GasA	97.808
MiscFeature	4	NaN	96.301
LandSlope	3	Gtl	94.658
Alley	2	NaN	93.767
CentralAir	2	Y	93.493
Functional	7	Typ	93.151
PavedDrive	3	Y	91.781
Electrical	5	SBrkr	91.370
GarageCond	5	TA	90.822
GarageQual	5	TA	89.795
BsmtCond	4	TA	89.795
LandContour	4	Lvl	89.795
ExterCond	5	TA	87.808
SaleType	9	WD	86.781
Condition1	9	Norm	86.301
BsmtFinType2	6	Unf	86.027
BldgType	5	1Fam	83.562
SaleCondition	6	Normal	82.055
Fence	4	NaN	80.753
MSZoning	5	RL	78.836
RoofStyle	6	Gable	78.151
LotConfig	5	Inside	72.055
BsmtExposure	4	No	65.274
LotShape	4	Reg	63.356
ExterQual	4	TA	62.055
MasVnrType	3	NaN	59.726

```
GarageType            6   Attchd           59.589
HeatingQC             5       Ex           50.753
KitchenQual           4       TA           50.342
HouseStyle            8   1Story           49.726
FireplaceQu           5      NaN           47.260
BsmtQual              4       TA           44.452
Foundation            6    PConc           44.315
GarageFinish          3      Unf           41.438
Exterior1st          15  VinylSd           35.274
Exterior2nd          16  VinylSd           34.521
BsmtFinType1          6      Unf           29.452
Neighborhood         25    NAmes           15.411
```

```
# ===== Bar chart tâ`n sô´ cho từng biê´n categorical (tô´i đa 10 nhóm phô’ biê´n nhâ´t) =====
if cat_cols:
    n = len(cat_cols)
    ncols = 4
    nrows = int(np.ceil(n / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(4.5 * ncols, 3 * nrows))
    axes = np.atleast_1d(axes).ravel()
    for ax, c in zip(axes, cat_cols):
        vc = train[c].fillna("Missing").value_counts().head(10)
        sns.barplot(x=vc.index.astype(str), y=vc.values, ax=ax)
        ax.set_title(c, fontsize=9)
        ax.tick_params(axis="x", rotation=60, labelsize=7)
    for ax in axes[n:]:
        ax.axis("off")
    plt.tight_layout()
    plt.show()
```


MSZoning
RL
RM
FV
RH
C (all)
MSZoning
Street
Pave
Grvl
Street
Alley
Missing
Grvl
Pave
Alley
LotShape
Reg
IR1
IR2
IR3
LotShape
LandContour
Lvl
Bnk
HLS
Low
LandContour
Utilities
AllPub
NoSeWa
Utilities
LotConfig
Inside
Corner
CulDSac
FR2
FR3
LotConfig
LandSlope
Gtl
Mod
Sev
LandSlope
Neighborhood
NAmes
CollgCr
OldTown
Edwards
Somerst
Gilbert
NridgHt
Sawyer
NWAmes
SawyerW
Neighborhood
Condition1
Norm
Feedr
Artery
RRAn
PosN
RRAe
PosA
RRNn
RRNe
Condition1
Condition2
Norm
Feedr
Artery
RRNn
PosN
PosA
RRAn
RRAe
Condition2
BldgType
1Fam
TwnhsE
Duplex
Twnhs
2fmCon
BldgType
HouseStyle
1Story
2Story
1.5Fin
SLvl
SFoyer
1.5Unf
2.5Unf
2.5Fin
HouseStyle
RoofStyle
Gable
Hip
Flat
Gambrel
Mansard
Shed
RoofStyle
RoofMatl
CompShg
Tar&Grv
WdShngl
WdShake
Metal
Membran
Roll
ClyTile
RoofMatl
Exterior1st
VinylSd
HdBoard
MetalSd
Wd Sdng
Plywood
CemntBd
BrkFace
WdShing
Stucco
AsbShng
Exterior1st
Exterior2nd
VinylSd
MetalSd
HdBoard
Wd Sdng
Plywood
CmentBd
Wd Shng
Stucco
BrkFace
AsbShng
Exterior2nd
MasVnrType
Missing
BrkFace
Stone
BrkCmn
MasVnrType
ExterQual
TA
Gd
Ex
Fa
ExterQual
ExterCond
TA
Gd
Fa
Ex
Po
ExterCond
Foundation
PConc
CBlock
BrkTil
Slab
Stone
Wood
Foundation
BsmtQual
TA
Gd
Ex
Missing
Fa
BsmtQual
BsmtCond
TA
Gd
Fa
Missing
Po
BsmtCond
BsmtExposure
No
Av
Gd
Mn
Missing
BsmtExposure
BsmtFinType1
Unf
GLQ
ALQ
BLQ
Rec
LwQ
Missing
BsmtFinType1
BsmtFinType2
Unf
Rec
LwQ
Missing
BLQ
ALQ
GLQ
BsmtFinType2
Heating
GasA
GasW
Grav
Wall
OthW
Floor
Heating
HeatingQC
Ex
TA
Gd
Fa
Po
HeatingQC
CentralAir
Y
N
CentralAir
Electrical
SBrkr
FuseA
FuseF
FuseP
Mix
Missing
Electrical
KitchenQual
TA
Gd
Ex
Fa
KitchenQual
Functional
Typ
Min2
Min1
Mod
Maj1
Maj2
Sev
Functional
FireplaceQu
GarageType
GarageFinish
GarageQual

3. Phân tích đa biến *(Mục 2.6 và 2.7)*

Mục tiêu: tìm **mối quan hệ** giữa các biến với giá nhà, và giữa các biến với nhau (để phát hiện biến dư thừa – *redundant*). Công cụ: covariance, hệ số tương quan Pearson (Eq. 2.18), heatmap, scatter plot, pairplot, Chi-square.

3.1 Tương quan giữa các biến numeric và giá nhà

```
# ===== Tương quan Pearson cuả từng biế´n numeric với giá nhà =====
# r > 0: cùng tăng; r < 0: một biế´n tăng thì biế´n kia giaỉm; r ~ 0:
không có quan hệ tuyế´n tính
corr_all = train[num_cols + [TARGET]].corr()
corr_target = corr_all[TARGET].drop(TARGET).sort_values(key=abs,
ascending=False)

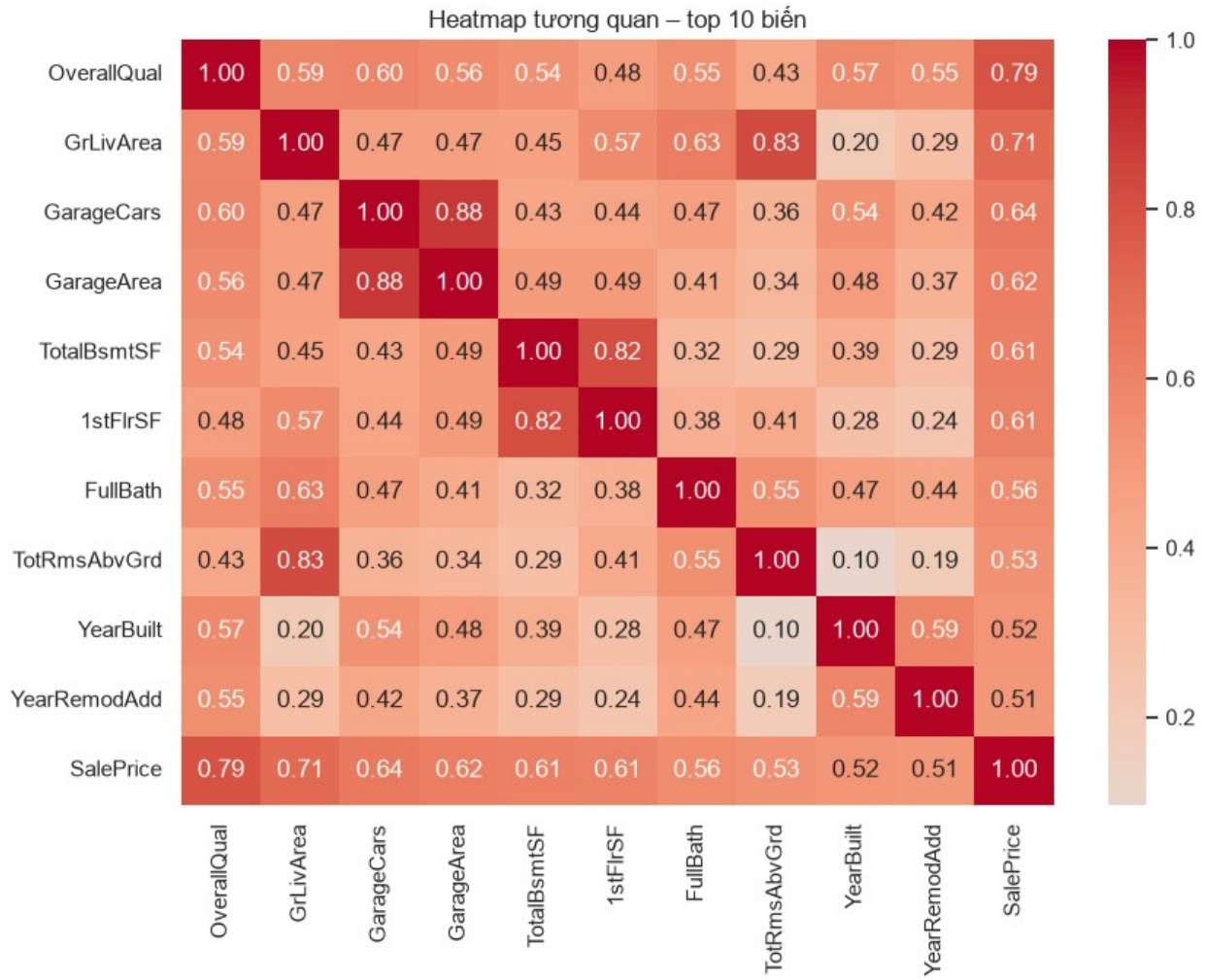
print("Top 15 biến tương quan mạnh nhất với", TARGET)
corr_target.head(15).to_frame("Pearson r")
```

```
Top 15 biến tương quan mạnh nhất với SalePrice
```

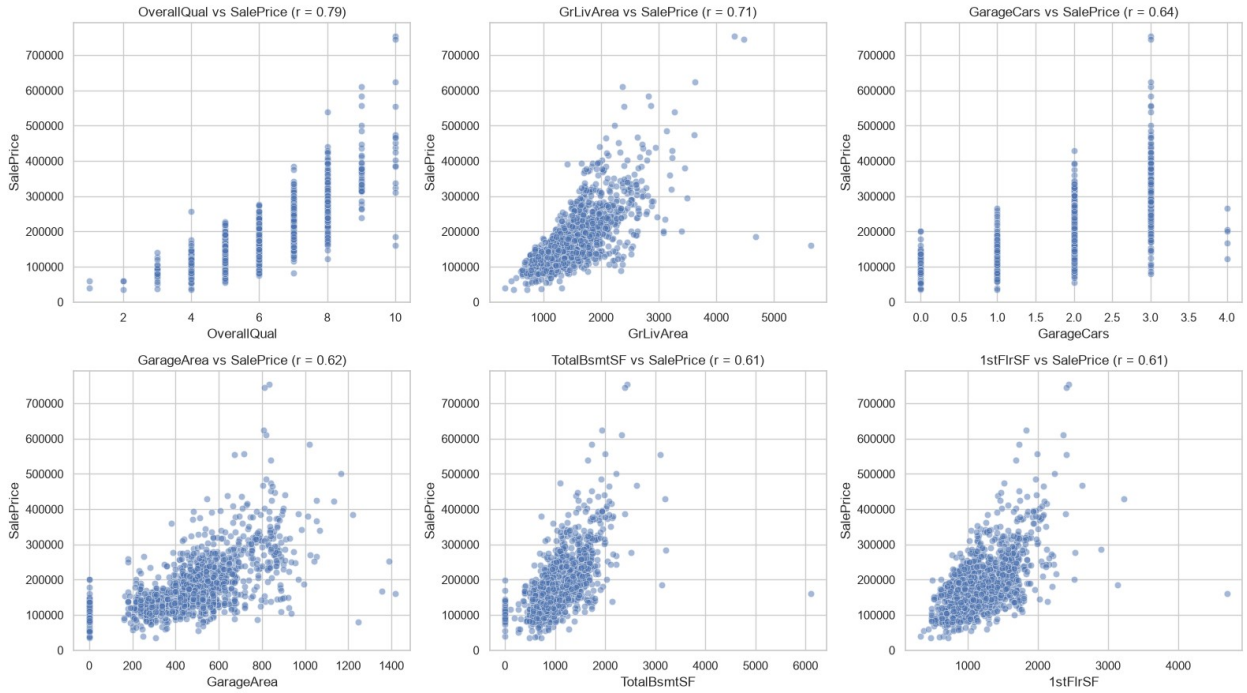
	Pearson r
OverallQual	0.791
GrLivArea	0.709
GarageCars	0.640
GarageArea	0.623
TotalBsmtSF	0.614
1stFlrSF	0.606
FullBath	0.561
TotRmsAbvGrd	0.534
YearBuilt	0.523
YearRemodAdd	0.507
GarageYrBlt	0.486
MasVnrArea	0.477
Fireplaces	0.467
BsmtFinSF1	0.386
LotFrontage	0.352

```
# ===== Heatmap tương quan cuả 10 biế´n mạnh nhấ´t =====
top_k = corr_target.head(10).index.tolist()

plt.figure(figsize=(9, 7))
sns.heatmap(train[top_k + [TARGET]].corr(), annot=True, fmt=".2f",
cmap="coolwarm", center=0)
plt.title("Heatmap tương quan – top 10 biến")
plt.tight_layout()
plt.show()
```




```
# ===== Scatter plot: 6 biến mạnh nhất so với giá nhà =====
# Scatter plot cho thấy độ mạnh, hình dạng, hướng của quan hệ và các outlier (Hình 2.11)
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
for ax, c in zip(axes.ravel(), corr_target.head(6).index):
    sns.scatterplot(x=train[c], y=train[TARGET], ax=ax, alpha=0.5)
    ax.set_title(f"{c} vs {TARGET} (r = {corr_target[c]:.2f})")
plt.tight_layout()
plt.show()
```




```
# ===== Ma trận covariance cuả 5 biế´n mạnh nhấ´t (Eq. 2.17) =====
# Covariance cho biế´t hướng biế´n thiên chung nhưng phụ thuộc đơn vị đo;
# vì vậy người ta chuẩn hóa thành correlation để so sánh được.
train[top_k[:5] + [TARGET]].cov()
```

	OverallQual	GrLivArea	GarageCars	GarageArea	\
OverallQual	1.913	430.961	0.621	166.185	
GrLivArea	430.961	276,129.633	183.488	52,691.984	
GarageCars	0.621	183.488	0.558	141.002	
GarageArea	166.185	52,691.984	141.002	45,712.510	
TotalBsmtSF	326.303	104,861.241	142.479	45,647.917	
SalePrice	86,904.126	29,581,866.743	38,020.181	10,589,102.523	

	TotalBsmtSF	SalePrice
OverallQual	326.303	86,904.126
GrLivArea	104,861.241	29,581,866.743
GarageCars	142.479	38,020.181
GarageArea	45,647.917	10,589,102.523
TotalBsmtSF	192,462.362	21,384,416.734
SalePrice	21,384,416.734	6,311,111,264.297

3.2 Phát hiện biến dư thừa (tương quan cao giữa các đặc trưng)

```
# ===== Tìm các cặp đặc trưng có |r| > 0.8 =====
# Sách (mục 2.6.1): nế´u 2 chiề`u tương quan mạnh thì có thể bo bớt một chiề`u vì dư thừa
corr_feat = train[num_cols].corr()
```

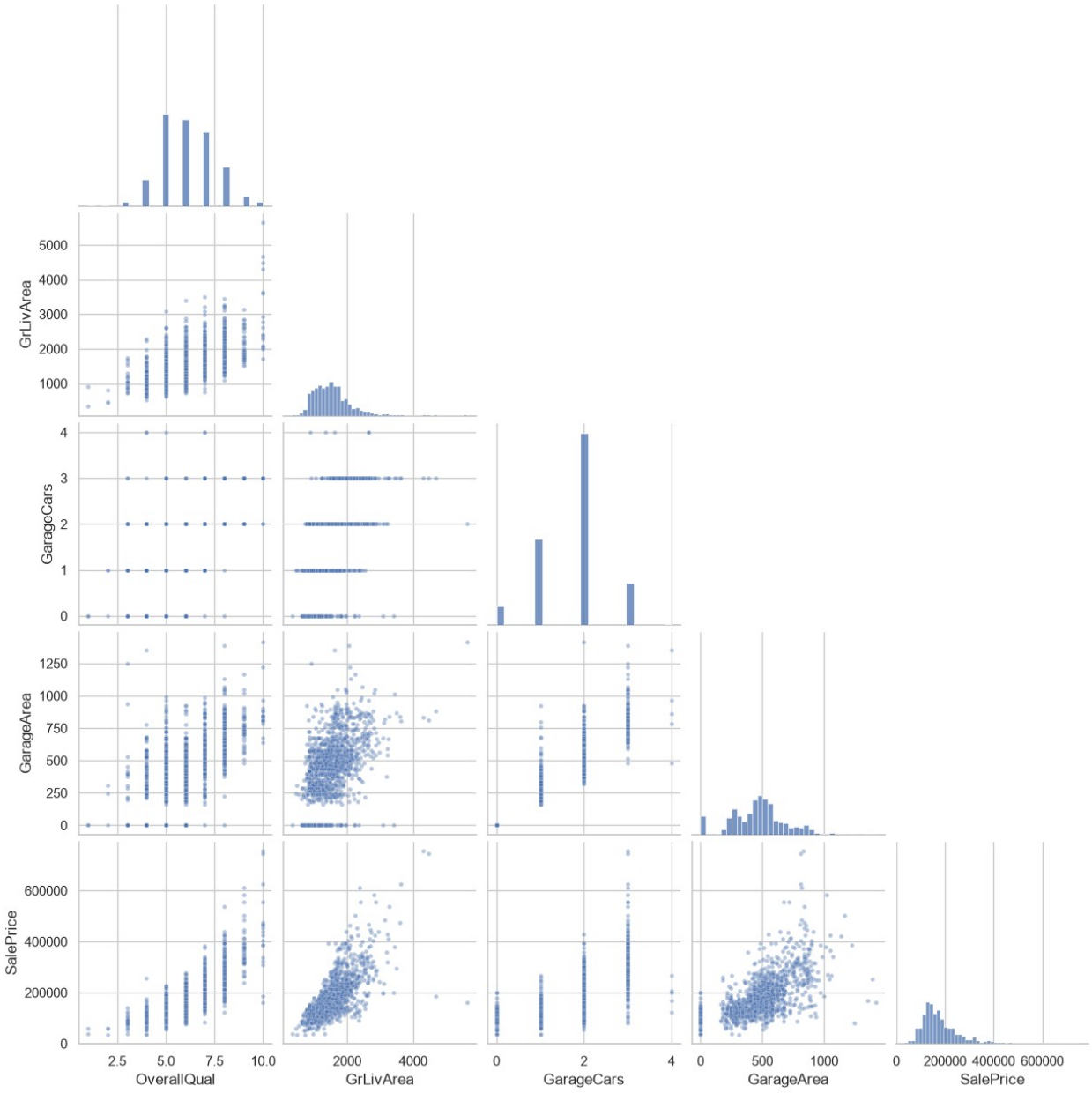
```
mask = np.triu(np.ones(corr_feat.shape), k=1).astype(bool)     # chỉ lấy nửa trên, tránh trùng cặp
pairs = corr_feat.where(mask).stack()
pairs = pairs[pairs.abs() > 0.8].sort_values(key=abs, ascending=False)

if len(pairs):
    display(pairs.to_frame("Pearson r"))
else:
    print("Không có cặp đặc trưng nào có |r| > 0.8")
```

		Pearson r
GarageCars	GarageArea	0.882
YearBuilt	GarageYrBlt	0.826
GrLivArea	TotRmsAbvGrd	0.825
TotalBsmtSF	1stFlrSF	0.820

3.3 Pairplot (ma trận scatter) *(Hình 2.14)*

```
# ===== Pairplot cho 4 biến mạnh nhất + giá nhà =====
# Đường chéo: phân phối từng biến; các ô còn lại: scatter của từng cặp biến
sns.pairplot(train[top_k[:4] + [TARGET]].dropna(), corner=True,
             plot_kws={"alpha": 0.4, "s": 12})
plt.show()
```



3.4 Biến categorical và giá nhà (Chi-square và boxplot)

```
# ===== Kiểm định Chi-square (Mục 2.9.1) =====
# H0: biến categorical KHÔNG liên quan đến nhóm giá nhà.
# Chia giá nhà thành 4 nhóm theo tứ phân vị, rồi kiểm tra độc lập
với từng biến categorical.
price_bin = pd.qcut(train[TARGET], 4, labels=False, duplicates="drop")

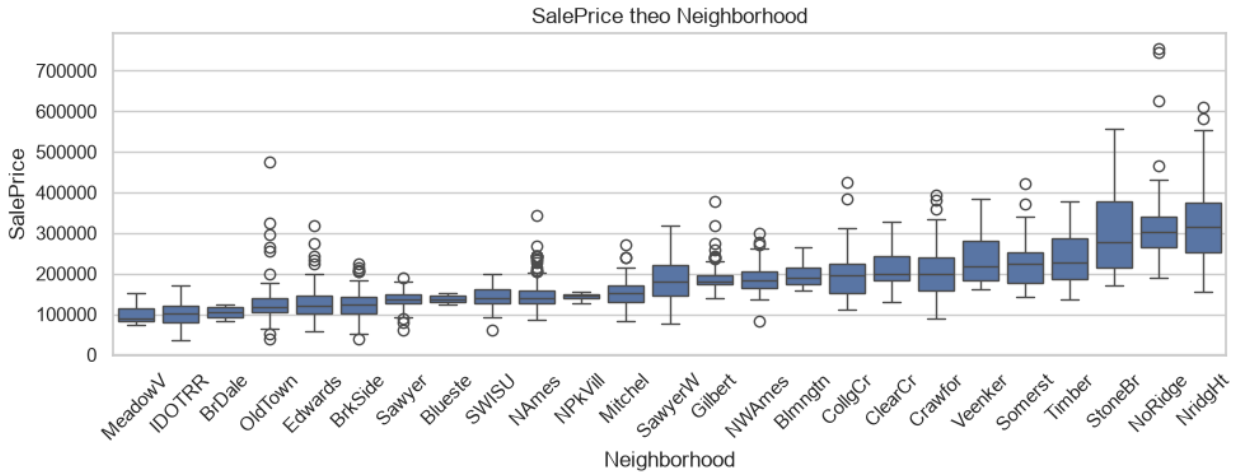
chi_rows = []
for c in cat_cols:
    ct = pd.crosstab(train[c].fillna("Missing"), price_bin)    # bảng
tần số quan sát (observed)
    chi2, p, dof, _ = stats.chi2_contingency(ct)               # tính
```

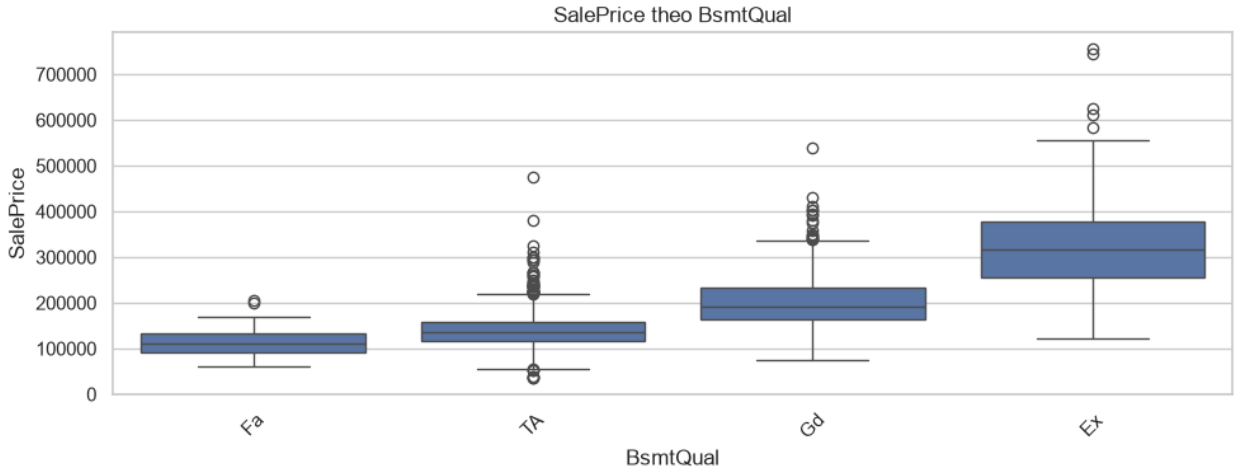
```
thô´ng kê chi2 và p-value
    chi_rows.append({"feature": c, "chi2": chi2, "dof": dof,
"p_value": p,
                     "có liên quan (p<0.05)": p < 0.05})

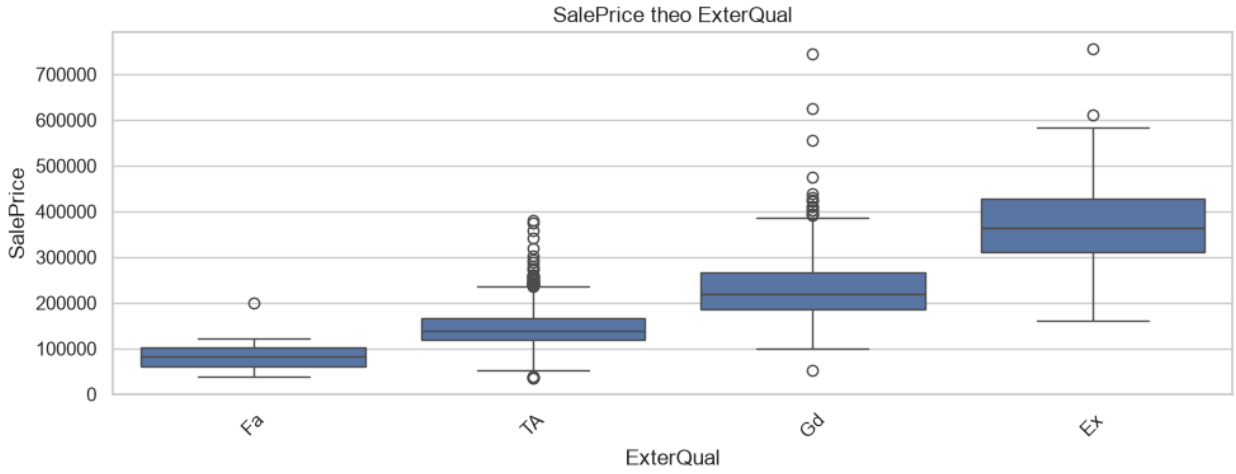
if chi_rows:
    chi_df =
pd.DataFrame(chi_rows).set_index("feature").sort_values("chi2",
ascending=False)
    display(chi_df.head(15))
```

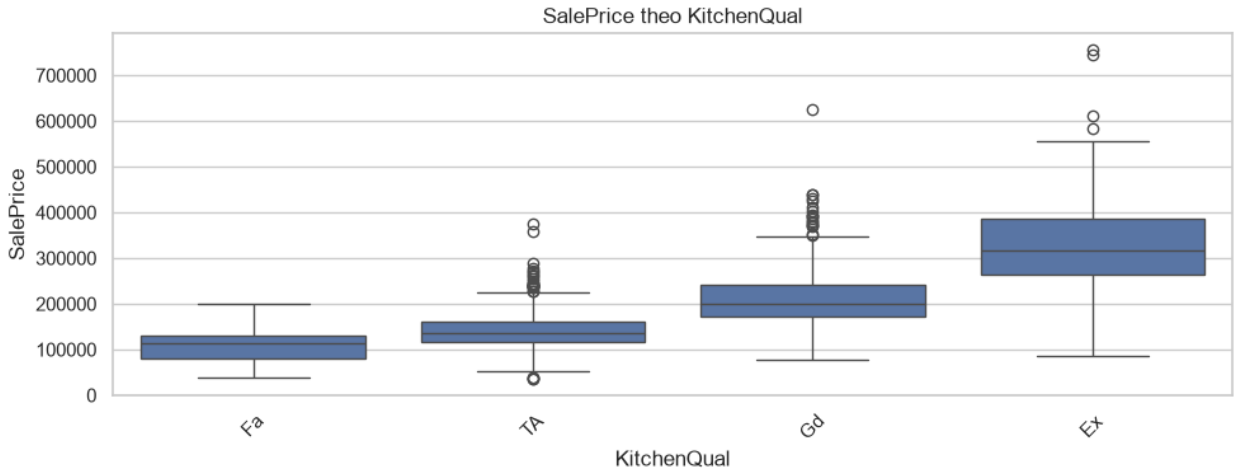
feature	chi2	dof	p_value	có liên quan (p<0.05)
Neighborhood	1,214.306	72	0.000	True
BsmtQual	779.015	12	0.000	True
ExterQual	725.401	9	0.000	True
KitchenQual	722.058	9	0.000	True
GarageFinish	658.819	9	0.000	True
GarageType	585.582	18	0.000	True
Foundation	517.647	15	0.000	True
FireplaceQu	495.615	15	0.000	True
BsmtFinType1	400.883	18	0.000	True
Exterior1st	377.526	42	0.000	True
HeatingQC	360.421	12	0.000	True
Exterior2nd	357.004	45	0.000	True
MSZoning	309.013	12	0.000	True
MasVnrType	274.932	9	0.000	True
HouseStyle	251.070	21	0.000	True

```
# ===== Boxplot giá nhà theo 4 biê´n categorical liên quan nhâ´t =====
if chi_rows:
    for c in chi_df.head(4).index:
        order = train.groupby(c)[TARGET].median().sort_values().index
# sắ´p xê´p nhóm theo median giá
        plt.figure(figsize=(10, 4))
        sns.boxplot(x=train[c].fillna("Missing"), y=train[TARGET],
order=list(order))
        plt.xticks(rotation=45)
        plt.title(f"{TARGET} theo {c}")
        plt.tight_layout()
        plt.show()
```


SalePrice theo Neighborhood
SalePrice
0
100000
200000
300000
400000
500000
600000
700000
MeadowV
IDOTRR
BrDale
OldTown
Edwards
BrkSide
Sawyer
Blueste
SWISU
NAmes
NPkVill
Mitchel
SawyerW
Gilbert
NWAmes
Blmngtn
CollgCr
ClearCr
Crawfor
Veenker
Somerst
Timber
StoneBr
NoRidge
NridgHt
Neighborhood


SalePrice theo BsmtQual
SalePrice
0
100000
200000
300000
400000
500000
600000
700000
Fa
TA
Gd
Ex
BsmtQual


SalePrice theo ExterQual
SalePrice
0
100000
200000
300000
400000
500000
600000
700000
Fa
TA
Gd
Ex
ExterQual



Nhận xét (điền sau khi chạy):

- Các biến có tương quan mạnh nhất với giá: …
- Các cặp biến dư thừa (|r| > 0.8): … → có thể giữ một biến.
- Các biến categorical liên quan nhất đến giá: …

  Nhớ rằng **tương quan không có nghĩa là nhân quả**: sách có đoạn viết bivariate analysis "deals with causes", nhưng hệ số tương quan chỉ đo mức liên hệ tuyến tính.

4. Giá trị thiếu và mẫu bất thường *(Mục 2.3.2 – Data Preprocessing)*

Sách gọi dữ liệu thực tế là **'dirty'**: thiếu giá trị (missing), không nhất quán (inconsistent), nhiễu (noisy), ngoại lệ (outlier), trùng lặp (duplicate).

4.1 Giá trị thiếu

```
# ===== Thống kê giá trị thiếu ở train và test =====
test_aligned = test.reindex(columns=train.columns)      # căn cột của
test giống train để so sánh
miss = pd.DataFrame({
    "số thiếu (train)": train.isna().sum(),
    "tỷ lệ (train, %)": train.isna().mean() * 100,
    "số thiếu (test)": test_aligned.isna().sum(),
    "tỷ lệ (test, %)": test_aligned.isna().mean() * 100,
})
# Chỉ giữ cột có thiếu, sắp xếp từ nhiều đến ít
miss = miss[(miss["số thiếu (train)"] > 0) | (miss["số thiếu (test)"]
> 0)] \
        .sort_values("tỷ lệ (train, %)", ascending=False)

print(f"Số cột có giá trị thiếu: {len(miss)} / {train.shape[1]}")
miss
```

```
Số cột có giá trị thiếu: 35 / 81
```

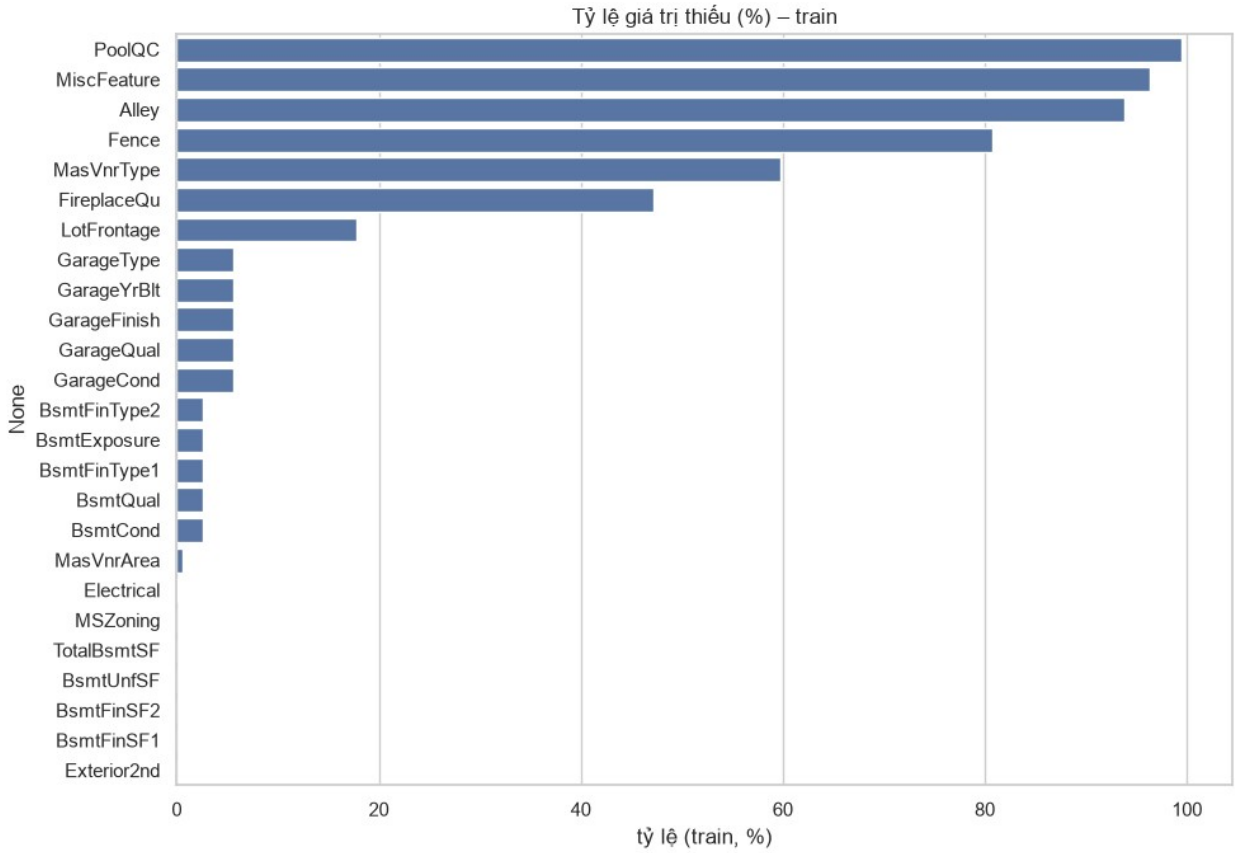
	số thiếu (train)	tỷ lệ (train, %)	số thiếu (test)	tỷ lệ (test, %)
PoolQC	1453	99.521	1456	99.794
MiscFeature	1406	96.301	1408	96.504
Alley	1369	93.767	1352	92.666
Fence	1179	80.753	1169	80.123
MasVnrType	872	59.726	894	61.275
FireplaceQu	690	47.260	730	50.034
LotFrontage	259	17.740	227	15.559
GarageType	81	5.548	76	5.209
GarageYrBlt	81	5.548	78	5.346
GarageFinish	81	5.548	78	5.346
GarageQual	81	5.548	78	5.346
GarageCond	81	5.548	78	5.346
BsmtFinType2	38	2.603	42	2.878
BsmtExposure	38	2.603	44	3.015
BsmtFinType1	37	2.534	42	2.878
BsmtQual	37	2.534	44	3.015
BsmtCond	37	2.534	45	3.083
MasVnrArea	8	0.548	15	1.028
Electrical	1	0.068	0	0.000
MSZoning	0	0.000	4	0.274
TotalBsmtSF	0	0.000	1	0.068
BsmtUnfSF	0	0.000	1	0.068
BsmtFinSF2	0	0.000	1	0.068
BsmtFinSF1	0	0.000	1	0.068
Exterior2nd	0	0.000	1	0.068
Utilities	0	0.000	2	0.137
Exterior1st	0	0.000	1	0.068
BsmtFullBath	0	0.000	2	0.137
BsmtHalfBath	0	0.000	2	0.137
KitchenQual	0	0.000	1	0.068
Functional	0	0.000	2	0.137
GarageCars	0	0.000	1	0.068
GarageArea	0	0.000	1	0.068
SaleType	0	0.000	1	0.068
SalePrice	0	0.000	1459	100.000

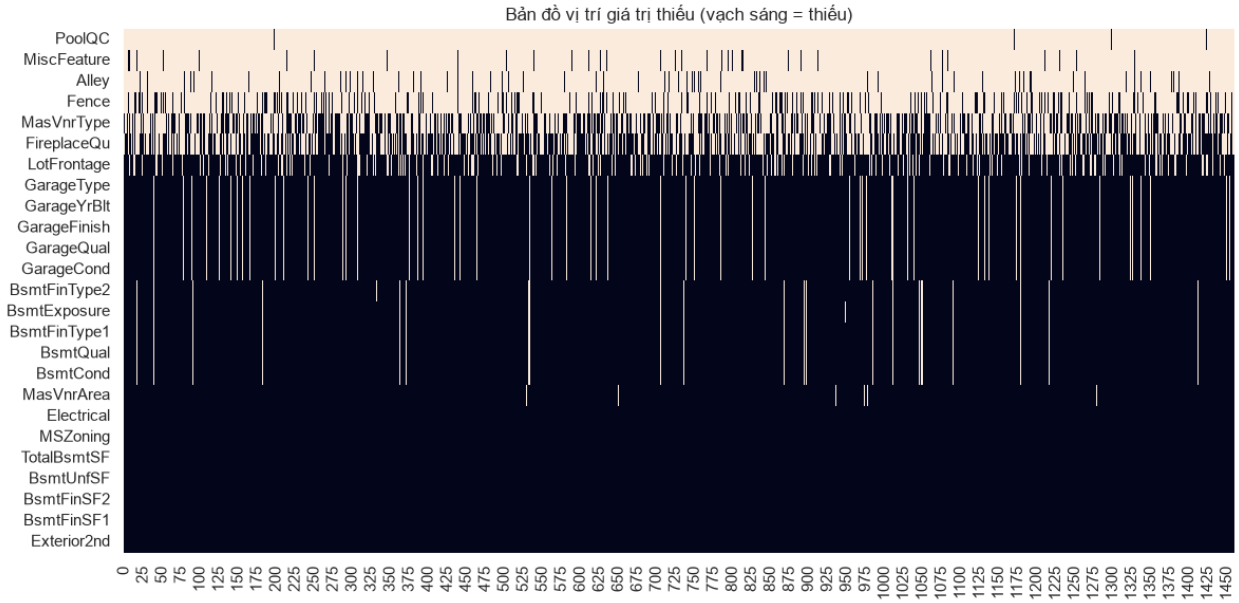
```
BsmtFinType2              2.879
BsmtExposure              3.016
BsmtFinType1              2.879
BsmtQual                  3.016
BsmtCond                  3.084
MasVnrArea                1.028
Electrical                0.000
MSZoning                  0.274
TotalBsmtSF               0.069
BsmtUnfSF                 0.069
BsmtFinSF2                0.069
BsmtFinSF1                0.069
Exterior2nd               0.069
Utilities                 0.137
Exterior1st               0.069
BsmtFullBath              0.137
BsmtHalfBath              0.137
KitchenQual               0.069
Functional                0.137
GarageCars                0.069
GarageArea                0.069
SaleType                  0.069
SalePrice               100.000
```

```
# ===== Trực quan hóa giá trị thiếu =====
if len(miss):
    top_miss = miss.head(25)

    # Biểu đồ cột: tỷ lệ thiếu của từng cột
    plt.figure(figsize=(10, 7))
    sns.barplot(x=top_miss["tỷ lệ (train, %)"], y=top_miss.index)
    plt.title("Tỷ lệ giá trị thiếu (%) — train")
    plt.tight_layout()
    plt.show()

    # Bản đồ vị trí thiếu: vạch sáng = ô bị thiếu. Giúp thấy các
cột thiếu cùng lúc
    # (vd: các cột về garage thường cùng thiếu ở những căn nhà
không có garage)
    plt.figure(figsize=(12, 6))
    sns.heatmap(train[top_miss.index].isna().T, cbar=False,
yticklabels=True)
    plt.title("Bản đồ vị trí giá trị thiếu (vạch sáng = thiếu)")
    plt.tight_layout()
    plt.show()
```





Cách xử lý giá trị thiếu (theo sách và thực tế):

Tình huống	Cách xử lý
Thiếu rất nhiều (> 40–50%)	Cân nhắc bỏ cột, hoặc hiểu là "không có tính năng đó"

Tình huống	Cách xử lý
Thiếu vì *không có* (vd: `PoolQC` thiếu = nhà không có hồ bơi)	Điền `"None"`
Biến numeric	Điền **median** (ít nhạy với outlier hơn mean)
Biến categorical	Điền **mode** hoặc `"Unknown"`

Lưu ý: điền giá trị luôn tiềm ẩn **bias** (sai lệch), vì giá trị điền chỉ là ước lượng.

4.2 Mẫu bất thường và mâu thuẫn

```
# ===== Quét các dấu hiệu dữ liệu bất thường =====
issues = []

# (a) Giá trị âm ở cột không thể âm (giống ví dụ Salary = -1500
trong Table 2.1)
for c in num_cols:
    neg = (train[c] < 0).sum()
    if neg:
        issues.append(f"{c}: {neg} giá trị âm")
if (train[TARGET] <= 0).any():
    issues.append(f"{TARGET}: có giá trị <= 0")

# (b) Mâu thuẫn logic giữa các cột (giống ví dụ tuổi 5 nhưng sinh
năm 1980: inconsistent data)
# Mỗi bộ: (cột A, cột B, 'A lớn hơn B là sai' hay 'A nhỏ hơn B là
sai', thông báo)
logic_checks = [
    ("YearBuilt", "YrSold", "greater", "Năm xây > năm bán"),
    ("YearRemodAdd", "YearBuilt", "less", "Năm cải tạo < năm xây"),
    ("GarageYrBlt", "YearBuilt", "less", "Năm xây garage < năm xây
nhà"),
]
for a, b, mode, msg in logic_checks:
    if a in train.columns and b in train.columns:
        bad = (train[a] > train[b]).sum() if mode == "greater" else
(train[a] < train[b]).sum()
        if bad:
            issues.append(f"{msg}: {bad} dòng ({a} vs {b})")

# (c) Cột gần như hằng số (>95% cùng một giá trị) -> mang rất ít
thông tin
for c in feature_cols:
    top_ratio = train[c].value_counts(normalize=True,
dropna=False).iloc[0]
    if top_ratio > 0.95:
        issues.append(f"{c}: {top_ratio:.1%} là cùng một giá trị (ít
thông tin)")

# (d) Phân phối train khác test (kiểm định Kolmogorov-Smirnov): có
```

```
thể gây lệch khi dự đoán
for c in num_cols:
    if c in test.columns and test[c].notna().sum() > 30:
        ks = stats.ks_2samp(train[c].dropna(), test[c].dropna())
        if ks.pvalue < 0.01:
            issues.append(f"{c}: phân phối train và test khác nhau (KS
p={ks.pvalue:.3g})")

print("\n".join(issues) if issues else "Không phát hiện bất thường rõ
ràng.")
```

```
Năm xây garage < năm xây nhà: 9 dòng (GarageYrBlt vs YearBuilt)
Street: 99.6% là cùng một giá trị (ít thông tin)
Utilities: 99.9% là cùng một giá trị (ít thông tin)
Condition2: 99.0% là cùng một giá trị (ít thông tin)
RoofMatl: 98.2% là cùng một giá trị (ít thông tin)
Heating: 97.8% là cùng một giá trị (ít thông tin)
LowQualFinSF: 98.2% là cùng một giá trị (ít thông tin)
KitchenAbvGr: 95.3% là cùng một giá trị (ít thông tin)
3SsnPorch: 98.4% là cùng một giá trị (ít thông tin)
PoolArea: 99.5% là cùng một giá trị (ít thông tin)
PoolQC: 99.5% là cùng một giá trị (ít thông tin)
MiscFeature: 96.3% là cùng một giá trị (ít thông tin)
MiscVal: 96.4% là cùng một giá trị (ít thông tin)
```

5. Xác định ngoại lệ (outlier) *(Mục 2.3.2 và 2.5.3)*

Hai phương pháp trong sách:

- **IQR:** điểm nằm ngoài khoảng `[Q1 − 1.5·IQR, Q3 + 1.5·IQR]` là ngoại lệ.
- **Z-score:** `|z| > 3` là ngoại lệ có thể (hạn chế: z-score nhạy với chính outlier vì dùng mean).

```
# ===== Đếm outlier theo cả hai phương pháp cho mỗi cột numeric
=====
out_rows = []
iqr_flags = pd.DataFrame(index=train.index)     # đánh dấu từng ô có
phải outlier không

for c in num_cols + [TARGET]:
    s = train[c].dropna()
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr          # hàng rào
dưới và trên

    flag_iqr = (train[c] < low) | (train[c] > high)     # outlier theo
IQR
    z = (train[c] - s.mean()) / s.std()                 # z-score (Eq.
2.2)
```

```
    flag_z = z.abs() > 3                                    # outlier theo
z-score

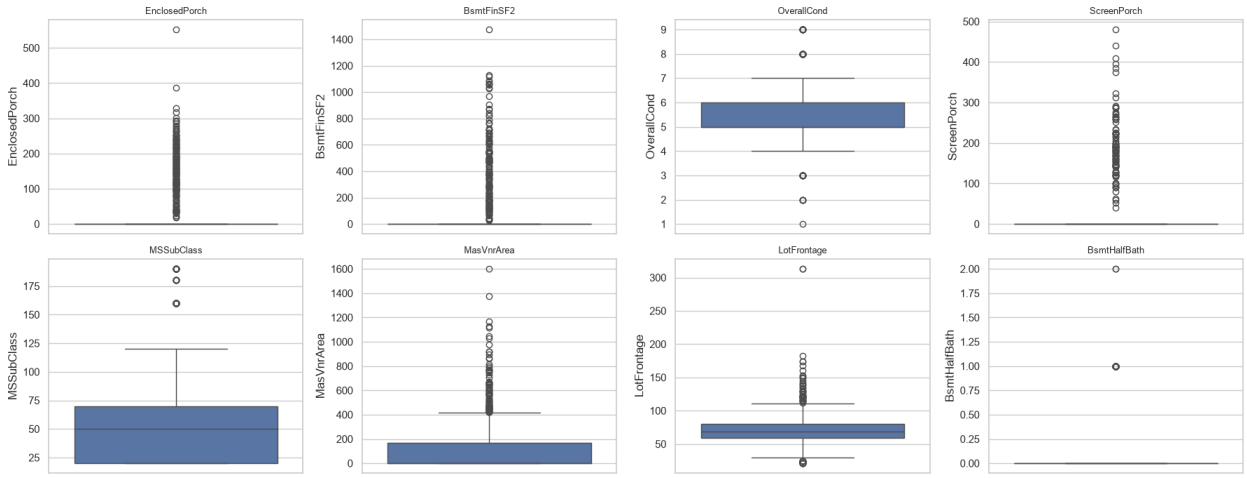
    iqr_flags[c] = flag_iqr
    out_rows.append({
        "feature": c,
        "hàng rào dưới": low,
        "hàng rào trên": high,
        "sô´outlier (IQR)": int(flag_iqr.sum()),
        "% outlier (IQR)": flag_iqr.mean() * 100,
        "sô´outlier (|z|>3)": int(flag_z.sum()),
    })

outliers = pd.DataFrame(out_rows).set_index("feature").sort_values("sô´
outlier (IQR)", ascending=False)
outliers.head(15)
```

feature	hàng rào dưới	hàng rào trên	sô´outlier (IQR)	% outlier (IQR)	sô´outlier (|z|>3)
EnclosedPorch	0.000	0.000	208	14.247	51
BsmtFinSF2	0.000	0.000	167	11.438	50
OverallCond	3.500	7.500	125	8.562	28
ScreenPorch	0.000	0.000	116	7.945	55
MSSubClass	-55.000	145.000	103	7.055	30
MasVnrArea	-249.000	415.000	96	6.575	32
LotFrontage	27.500	111.500	88	6.027	12
BsmtHalfBath	0.000	0.000	82	5.616	82
OpenPorchSF	-102.000	170.000	77	5.274	27
LotArea	1,481.500	17,673.500	69	4.726	13
KitchenAbvGr	1.000	1.000	68	4.658	68
SalePrice	3,937.500	340,037.500	61	4.178	22
TotalBsmtSF	42.000	2,052.000	61	4.178	10
MiscVal	0.000	0.000	52		
BedroomAbvGr	0.500	4.500	35		

```
MiscVal                  3.562                    8
BedroomAbvGr             2.397                   14
```

```
# ===== Boxplot cuả 8 biến có nhiều outlier nhất (chấm tròn ngoài râu = outlier) =====
top_out = outliers.head(8).index.tolist()
fig, axes = plt.subplots(2, 4, figsize=(18, 7))
for ax, c in zip(axes.ravel(), top_out):
    sns.boxplot(y=train[c], ax=ax)
    ax.set_title(c, fontsize=9)
plt.tight_layout()
plt.show()
```




```
# ===== Những dòng 'đáng ngờ': là outlier ở nhiều cột cùng lúc =====
n_flags = iqr_flags.drop(columns=TARGET).sum(axis=1)
top_rows = n_flags.sort_values(ascending=False).head(10).index
pd.DataFrame({"số cột outlier": n_flags[top_rows], TARGET:
train.loc[top_rows, TARGET]})
```

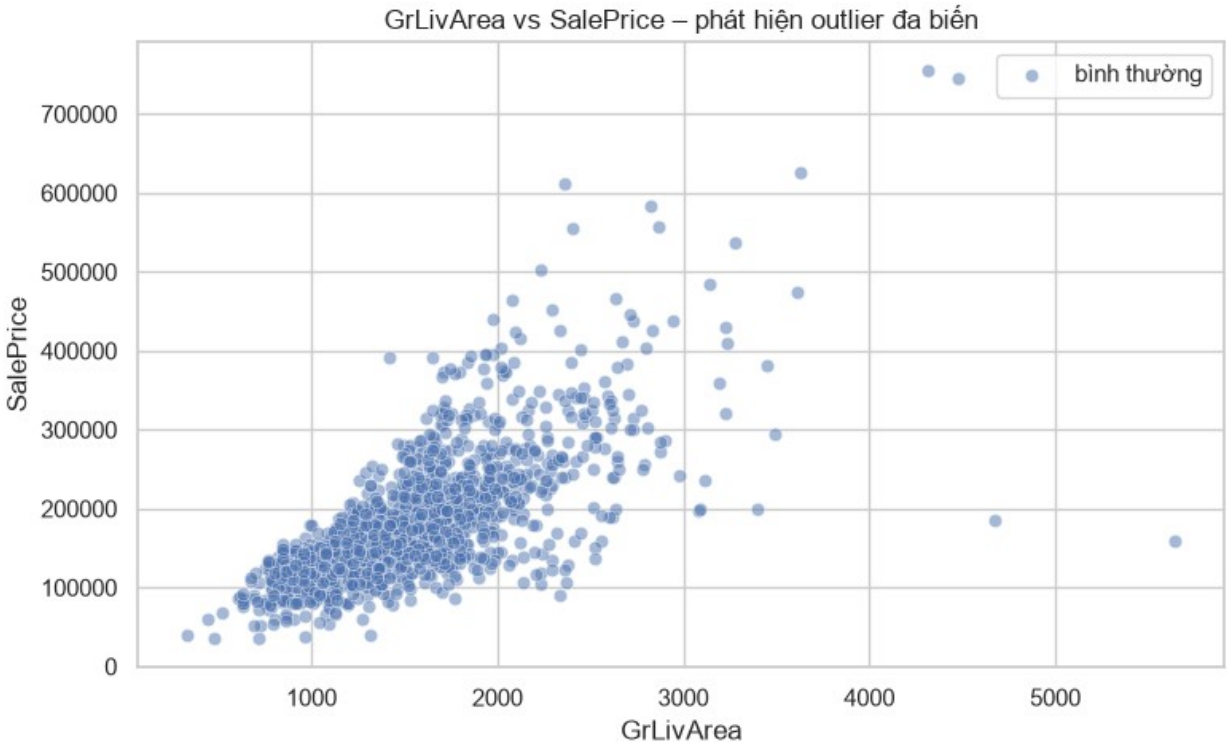
	số cột outlier	SalePrice
1298	12	160000
523	9	184750
197	8	235000
1386	7	250000
1182	7	745000
691	7	755000
185	7	475000
1230	6	190000
803	6	582933
1173	6	200500

```
# ===== Outlier đa biến: diện tích rất lớn nhưng giá lại thấp =====
# Một điểm có thể bình thường ở từng biến riêng lẻ nhưng bất thường khi xét hai biến cùng lúc.
area_col = "GrLivArea" if "GrLivArea" in train.columns else top_k[0]
```

```
hi_area = train[area_col].quantile(0.99)       # ngưỡng diện tích rất lớn (top 1%)
lo_price = train[TARGET].quantile(0.30)        # ngưỡng giá thấp (dưới 30%)
odd = train[(train[area_col] > hi_area) & (train[TARGET] < lo_price)]

plt.figure(figsize=(8, 5))
sns.scatterplot(x=train[area_col], y=train[TARGET], alpha=0.5, label="bình thường")
if len(odd):
    sns.scatterplot(x=odd[area_col], y=odd[TARGET], color="red", s=90, label="nghi ngờ")
plt.title(f"{area_col} vs {TARGET} – phát hiện outlier đa biến")
plt.legend()
plt.tight_layout()
plt.show()

print(f"Số điểm nghi ngờ: {len(odd)}")
odd[[c for c in [ID_COL, area_col, TARGET] if c]]
```




```
Số điểm nghi ngờ: 0

Empty DataFrame
Columns: [Id, GrLivArea, SalePrice]
Index: []
```

Cách xử lý ngoại lệ:

- Outlier **có thể là dữ liệu thật** (biệt thự đắt tiền); sách cũng nhấn mạnh outlier đôi khi là thứ ta quan tâm → *không xóa mù quáng*.
- Chỉ xóa khi có dấu hiệu sai/mâu thuẫn (vd: diện tích cực lớn nhưng giá rất thấp, như các điểm đỏ ở trên).
- Có thể dùng **log transform** hoặc **winsorize** (kéo giá trị cực đoan về hàng rào IQR) thay vì xóa.

6. Minh họa tiền xử lý: chuẩn hóa và PCA *(Mục 2.3.2 và 2.10.3)*

Phần mở rộng (điểm cộng): áp dụng **Min-Max** (Eq. 2.1), **z-score** (Eq. 2.2) và **PCA** để xem có thể giảm bao nhiêu chiều.

```
# ===== Chuẩn hóa dữ liệu numeric =====
X = train[num_cols].copy()
X = X.fillna(X.median())       # PCA không chấp nhận NaN, nên điền
median tạm thời

# Min-Max: đưa mọi biến về khoảng [0, 1]
X_minmax = pd.DataFrame(MinMaxScaler().fit_transform(X),
columns=X.columns)
# Z-score: đưa mọi biến về trung bình 0, độ lệch chuẩn 1
X_z = pd.DataFrame(StandardScaler().fit_transform(X),
columns=X.columns)

print(f"Min-Max: min = {X_minmax.min().min():.1f}, max =
{X_minmax.max().max():.1f}")
print(f"Z-score: mean ≈ {X_z.mean().mean():.4f}, std ≈
{X_z.std().mean():.3f}")
```

```
Min-Max: min = 0.0, max = 1.0
Z-score: mean ≈ 0.0000, std ≈ 1.000
```

```
# ===== PCA và Scree plot (Hình 2.15) =====
# Phải chuẩn hóa trước khi PCA, nếu không biến có đơn vị lớn (diện
tích) sẽ lấn át các biến khác
pca = PCA().fit(X_z)
cum = np.cumsum(pca.explained_variance_ratio_)          # phương sai
tích lũy
k90 = int(np.argmax(cum >= 0.90) + 1)                   # số thành
phần cần để giữ 90% phương sai
print(f"Cần {k90} / {X.shape[1]} thành phần chính để giữ 90% phương
sai của dữ liệu.")

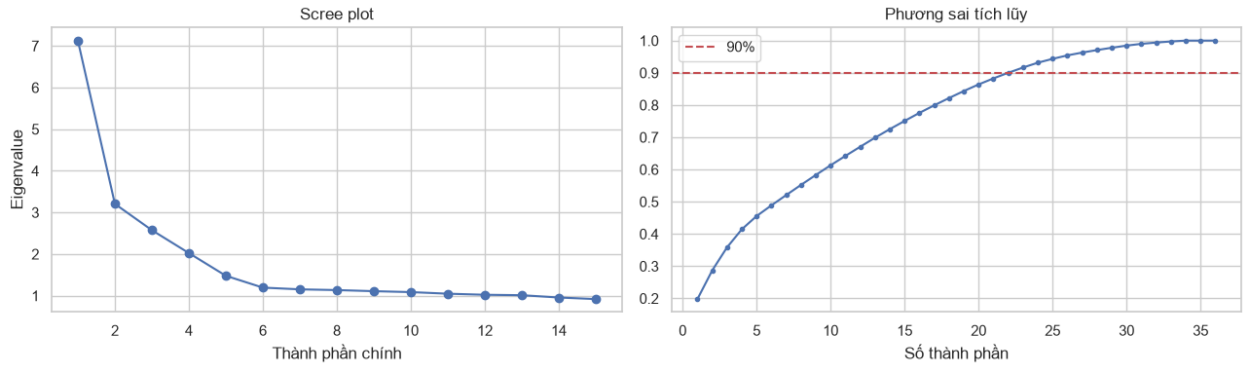
n_show = min(15, len(pca.explained_variance_))
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
```

```
# Scree plot: eigenvalue giảm dần; điểm 'khuỷu tay' cho biết nên
giữ bao nhiêu thành phần
axes[0].plot(range(1, n_show + 1), pca.explained_variance_[:n_show],
marker="o")
axes[0].set_title("Scree plot")
axes[0].set_xlabel("Thành phần chính")
axes[0].set_ylabel("Eigenvalue")

axes[1].plot(range(1, len(cum) + 1), cum, marker=".")
axes[1].axhline(0.9, color="r", ls="--", label="90%")
axes[1].set_title("Phương sai tích lũy")
axes[1].set_xlabel("Số thành phần")
axes[1].legend()

plt.tight_layout()
plt.show()
```

```
Cần 22 / 36 thành phần chính để giữ 90% phương sai của dữ liệu.
```



7. Kết luận và kế hoạch tiền xử lý

(Điền sau khi chạy xong – gợi ý khung:)

1. **Tính chất dữ liệu:** … (số mẫu, số biến, tỷ lệ numeric/categorical).
2. **Đơn biến:** giá nhà lệch phải (skewness = …) → dùng log-transform.
3. **Đa biến:** các biến quan trọng nhất là …; các cặp dư thừa là …
4. **Giá trị thiếu:** … cột bị thiếu; xử lý bằng median/mode/"None".
5. **Ngoại lệ:** phát hiện … điểm nghi ngờ (`GrLivArea` lớn nhưng giá thấp) → cân nhắc loại bỏ.
6. **Bước tiếp theo:** mã hóa biến categorical, chuẩn hóa, chọn đặc trưng (forward/backward selection hoặc PCA), huấn luyện mô hình hồi quy.